

Lab 01 — Performance Profiling and Optimization using GPROF and PERF

Sorting 50 years of daily temperature data: Bubble Sort vs Quick Sort

Harsh Raj

5 August 2026

1. Scenario

A weather analytics company stores roughly 50 years of daily temperature readings ($50 \times 365 = \mathbf{18,250}$ records) and sorts them. The sort is “slower than expected”. The task is to profile the existing baseline (**Bubble Sort**), compare it against the proposed replacement (**Quick Sort**), locate the bottleneck with GPROF and PERF, and make a recommendation.

Headline results

Question	Answer
Which algorithm is faster?	Quick Sort , by 187x at $N = 18,250$ and by 12,433x at $N = 1,000,000$
Bubble Sort @ 1,000,000 records	871.8 s (14 min 32 s) — <i>measured, not extrapolated</i>
Quick Sort @ 1,000,000 records	0.070 s
Top hotspot (Bubble, -00)	bubbleSort — 73.53% of CPU time
Second hotspot (Bubble, -00)	swap — 23.53%, over 82.9 million calls
Top hotspot (Quick, -00)	partition — 78.57% of CPU time
Best compiler setting	-02 — and -03 is 3.1x slower than -02 here (see §8)

The single most important number in this report: bubble sort does **166,521,222 comparisons** to sort 18,250 records; quicksort does **302,516**. That ratio — **550x** — is the whole story, and no amount of compiler optimization closes it.

2. Files

```
Lab01/
├── mysort.c           # the program: both sorts + timing harness
├── run_lab.sh         # reproduces every measurement in this report
├── README.md          # this document
├── report.pdf          # PDF rendering of this document
├── myreport.txt        # gprof report, -00
├── report_02.txt       # gprof report, -02
└── report_03.txt       # gprof report, -03
```

```

|— report_quick_4M.txt      # gprof report, quicksort-only at N=4,000,000
|— scaling.csv              # problem-size study, raw data
|— opt_sweep.csv           # optimization-level study, raw data
|— counters.txt            # exact comparison/swap counts
|— time_00/02/03.txt       # `time` output, 3 runs each
|— screenshots/            # terminal captures

```

Reproduce everything with:

```

./run_lab.sh                # ~3 minutes
./run_lab.sh --with-lm      # also runs bubble sort on 1,000,000 records (~15 min)

```

3. Environment

Item	Value
CPU	12th Gen Intel Core i7-1255U (6 cores / 12 threads)
Caches	L1d 48 KiB/core · L2 1.25 MiB/core (7.5 MiB total) · L3 12 MiB shared
Kernel	6.18.33.2-microsoft-standard-WSL2
GCC	11.4.0 (Ubuntu 11.4.0-1ubuntu1~22.04.3)
GNU gprof	2.38
perf	see §7 — hardware counters are unavailable on this kernel

Cache sizes matter later: 18,250 ints = **73 KB** (fits in L2), 1,000,000 ints = **4 MB** (fits in L3), 4,000,000 ints = **16 MB** (**exceeds** the 12 MB L3). This shows up directly in the scaling curve in §9.

4. Program design

`mysort.c` generates the dataset, then sorts **identical copies** of it with each algorithm and verifies the output is actually in order.

Data model. Temperatures are int in **millidegrees Celsius** ($21374 = 21.374\text{ }^{\circ}\text{C}$). Two reasons:

1. An int compare is a single instruction, so the profile measures the *sorting*, not floating-point conversion overhead.
2. $0.001\text{ }^{\circ}\text{C}$ resolution keeps duplicate keys rare. This is not cosmetic: the Lomuto partition used here degrades toward $O(n^2)$ on runs of equal keys, so a coarse resolution (say, whole degrees) would silently change what is being measured. At $0.001\text{ }^{\circ}\text{C}$ there are $\sim 40,000$ distinct keys.

The generator uses a triangular annual cycle ($3\text{ }^{\circ}\text{C}$ in mid-January to $27\text{ }^{\circ}\text{C}$ in mid-July) plus pseudo-random daily noise. It deliberately avoids `<math.h>` so the exact compile line from the lab sheet — `gcc -O0 -pg mysort.c -o mysort` — links without `-lm`.

Algorithms. `swap`, `partition` and `quickSort` are kept exactly as supplied in the brief, so the profile reflects the code under study. `swap` is deliberately a real function rather than a macro: at `-O0` it stays a genuine call and appears as a hotspot; at `-O2` it is inlined and vanishes. That contrast is one of the results this lab is meant to produce (§8).

Harness note. An earlier version of the harness dispatched through function pointers (bubble_adapter / quick_adapter). That was a mistake for a profiling lab: at -02/-03 the wrappers survive as symbols and the sort gets inlined *into* them, so gprof reported the hotspot as bubble_adapter — an artefact of the harness, not of the algorithm. It now dispatches through a switch, which inlines away cleanly.

5. Execution time

5.1 time ./mysort (N = 18,250, both sorts, best of 3)

Build	Real	User	Sys
-00 -pg	1.079 s	1.084 s	0.000 s
-02 -pg	0.186 s	0.183 s	0.004 s
-03 -pg	0.487 s	0.487 s	0.004 s

sys time is ~0 throughout: this is a pure compute workload with one up-front allocation and no I/O in the timed region. user $\approx$ real confirms it is single-threaded and CPU-bound, never blocked.

5.2 In-program timing (N = 18,250)

Build	Bubble (s)	Quick (s)	Speedup	Sorted
-00 -pg (profiling build)	1.0231	0.0027	381×	yes
-02, no -pg (best of 3)	0.174504	0.000932	187×	yes

The -02 row is the one to quote. The -pg row exaggerates the gap: -pg inflates bubble sort by 2.55× (\$6.5) while quicksort's ~1 ms sits close to the timer's noise floor, so its 381× is an artefact of the profiling build. Throughout this report, **-02 without -pg is the canonical configuration**; figures quoted from separate experiments may differ by a few percent, which is run-to-run variance.

5.3 A measurement trap: clock() under -pg

The first version of this harness reported quicksort's CPU time as **0.0000 s** while wall-clock said 0.0023 s. That is a measurement bug, not a free sort. A standalone test isolates it — the same 342 μ s workload:

=== WITHOUT -pg ===	=== WITH -pg ===
delta = 342 μ s	delta = 0 μ s
delta = 342 μ s	delta = 0 μ s

clock_getres() reports 1 ns for CLOCK_PROCESS_CPUTIME_ID in both builds, so this is not advertised resolution — clock() simply under-reports sub-millisecond intervals in -pg builds on this kernel. At coarser intervals (~4 ms) it agrees with getrusage and CLOCK_MONOTONIC to within 0.1%.

Consequence: this report uses CLOCK_MONOTONIC wall-clock as the primary metric, and - - repeat R reports the best of R runs. Best-of is the correct summary for a benchmark — noise can only ever make a run slower, never faster, so the minimum is the closest estimate of true cost.

6. GPROF analysis

6.1 Bubble Sort at -00 (myreport.txt) — the answer to “where is the time?”

% time	cumulative seconds	self seconds	calls	self ms/call	total ms/call	name
73.53	0.25	0.25	1	250.00	329.84	bubbleSort
23.53	0.33	0.08	82919103	0.00	0.00	swap
2.94	0.34	0.01	2	5.00	5.00	is_sorted
0.00	0.34	0.00	12273	0.00	0.00	partition

- **Hotspot: bubbleSort, 73.53% of CPU time.**
- **Second: swap, 23.53%, across 82,919,103 calls.** Individually swap is three instructions; collectively it is a quarter of the runtime, purely because it is *called 82.9 million times*. This is the classic profiling lesson: cost = unit cost × call count, and the call count dominates here.
- Everything else is noise. partition (all of quicksort’s real work) does not register at all.

6.2 Call graph

		0.12	0.03	1/1	run_one
[3]	91.1	0.12	0.03	1	bubbleSort [3]
		0.03	0.00	82757197/82919103	swap [5]

		0.00	0.00	161906/82919103	partition [7]
		0.03	0.00	82757197/82919103	bubbleSort [3]
[5]	20.6	0.04	0.00	82919103	swap [5]

				24546	quickSort [8]
[8]	0.0	0.00	0.00	1+24546	quickSort [8]
		0.00	0.00	12273/12273	partition [7]

Two things to read here:

1. **swap is shared.** The call graph splits its 82,919,103 calls by parent: **82,757,197 from bubbleSort** and **161,906 from partition** — a 511× difference from the same primitive.
2. **1+24546 is recursion.** gprof’s flat-profile calls column counts only *non-recursive* entries, which is why quickSort appears as “1” call there. The call graph shows the truth: 1 external call + **24,546 recursive calls**. Without reading the call graph you would wrongly conclude quicksort barely ran.

6.3 Cross-validation

The -DCOUNTERS build counts operations directly, independent of gprof’s mcount instrumentation. The two agree **exactly**:

Source	Bubble swaps	Quick swaps
gprof call graph	82,757,197	161,906
-DCOUNTERS	82,757,197	161,906

Two independent instruments giving identical counts is good evidence neither is lying. Full operation counts at N = 18,250:

Algorithm	Comparisons	Swaps
Bubble Sort	166,521,222	82,757,197
Quick Sort	302,516	161,906
Ratio	550x	511x

Theory check. Bubble sort's expected comparisons are $\approx n^2/2 = \mathbf{166,531,250}$ for $n = 18,250$; measured **166,521,222** — agreement to 4 significant figures. Quicksort's average case is $\approx 2n \cdot \ln n - 2.85n = \mathbf{306,100}$; measured **302,516**. Both algorithms behave exactly as the textbook analysis predicts, which is good evidence the implementations are correct.

6.4 Profiling quicksort — a subtlety worth knowing

At $N = 18,250$ quicksort finishes in ~ 1 ms, but **gprof samples every 10 ms**. Quicksort therefore collects *zero* samples and its self-time prints as 0.00 s — which does not mean it is free, only that it is below the profiler's resolution. **A profiler cannot measure what finishes faster than its sampling period.**

Re-profiling quicksort alone at $N = 4,000,000$ (report_quick_4M.txt) makes it measurable:

% time	cumulative seconds	self seconds	calls	self ms/call	total ms/call	name
78.57	0.33	0.33	3964001	0.00	0.00	partition
7.14	0.36	0.03	38647333	0.00	0.00	swap
4.76	0.40	0.02	1	20.00	380.00	quickSort

Quicksort's hotspot is partition at 78.57% — as theory predicts, since partition is where every comparison happens. Note also that gprof attributes a stray 2.38% to bubbleSort in this run *even though it was never called* (its calls column is blank) — a reminder that the time column is statistical sampling and carries error, while the call counts are exact instrumentation.

6.5 Profiling overhead

-pg is not free, and its cost depends entirely on how many *calls* there are:

Build	Without -pg	With -pg	Overhead
-00	0.394 s	1.004 s	2.55x
-02	0.177 s	0.173 s	$\sim 1.0\times$ (none)

At -00, mcount fires on all 82.7 M swap calls. At -02, swap is inlined, so there is almost nothing left to instrument. **Never quote a -pg build's absolute timings as the program's real performance** — §5.1's numbers are for comparison between builds only; the true timings are in §9.

7. Hardware performance counters (PERF)

7.1 This kernel has no PMU — with evidence

perf stat cannot report cycles, instructions, IPC, cache references, cache misses or branch misses on this machine, and installing perf does not fix it. The limitation is the virtualised kernel, not the tooling: WSL2 runs under Hyper-V, which exposes **no virtual PMU** to the guest.

Direct evidence — a perf_event_open() probe requesting each counter:

```

HW cpu-cycles      : No such file or directory (ENOENT)
HW instructions    : No such file or directory (ENOENT)
HW cache-references : No such file or directory (ENOENT)
HW branch-misses   : No such file or directory (ENOENT)
SW task-clock      : OK
SW page-faults     : OK

```

Every **hardware** event is rejected at the syscall; every **software** event works. `perf stat` would print <not supported> for exactly the rows the lab asks us to record.

To collect real hardware counters, run on native (non-virtualised) Linux:

```
perf stat -e task-clock,context-switches,page-faults,cycles,instructions,\
cache-references,cache-misses,branch-instructions,branch-misses ./mysort
```

7.2 Substitute measurements

Since the PMU is unavailable, the metrics were obtained by other means:

Metric	How it was obtained here
Execution time	CLOCK_MONOTONIC + time (§5)
Instructions	Cachegrind simulation + exact operation counts (§6.3)
Cache refs / misses	Cachegrind simulation
Branch misses	Cachegrind simulation
IPC	Not obtainable — requires a real cycle counter
Hotspot function	gprof (§6)

8. Compiler optimizations

Best of 9 runs, no `-pg`, wall-clock seconds:

Level	Bubble (N=18,250)	vs -O0	Quick (N=1,000,000)	vs -O0
-O0	0.337 s	1.00×	0.113 s	1.00×
-O1	0.165 s	2.04×	0.0548 s	2.06×
-O2	0.166 s	2.03×	0.0587 s	1.93×
-O3	0.520 s	0.65× (slower!)	0.0623 s	1.82×

8.1 What -O2 actually did

The gprof reports show the mechanism directly. At `-O0` the profile has four entries; at `-O2`:

```

100.00    0.18    0.18        1  180.00    180.00  bubbleSort
  0.00    0.18    0.00   12273    0.00     0.00  partition

```

swap has disappeared entirely — inlined into its callers. 82.9 million function calls, each with prologue, epilogue, stack frame and pointer indirection, became inline register operations. That single transformation is most of the 2× gain, and it is also why `-pg` overhead collapses to zero (§6.5).

At `-O3` the profile degrades further for profiling purposes: `bubbleSort` itself is inlined into `run_one`, so gprof attributes **100% to run_one** and the algorithm's own symbol is gone. **Aggressive inlining and profiling are in tension** — the more the compiler optimizes, the less the profile resembles the source. This is why production profiling usually uses `-O2 -g -fno-omit-frame-pointer` rather than `-O3`.

8.2 -03 is 3.1× slower than -02 — root cause

This is the most interesting result in the lab, and it is a real effect, not measurement noise or a -pg artefact. Isolating it:

Build	Bubble, N=18,250 (best of 5)
-02	0.171 s
-03	0.588 s
-03 -fno-tree-vectorize	0.205 s
-03 -fno-if-conversion	0.479 s

Disabling the vectorizer recovers nearly all the loss, which points straight at auto-vectorization. Confirmed in the generated assembly for bubbleSort:

	-02	-03
SIMD register refs (xmm/ymm)	0	6
Shuffle instructions (pshufd)	0	2

-03 enables -ftree-slp-vectorize, which tries to pack bubble sort's inner loop into SIMD registers. But bubble sort's inner loop carries a **loop-carried dependency** — `arr[j+1]` written on iteration j is read on iteration $j+1$ — so the vectorizer can only proceed by adding shuffles and extra data movement around it. The result is correct (output verified sorted) but the shuffle overhead costs more than the vector width saves.

Lesson: -03 is not “-02 but better”. It enables transformations that are profitable on average and harmful on specific code shapes. Always measure.

9. Effect of problem size

Wall-clock seconds, -02, no -pg (from scaling.csv):

N	Bubble Sort	Ratio	Quick Sort	Ratio
1,000	0.000424	—	0.000017	—
2,000	0.001894	4.47×	0.000061	3.59×
4,000	0.007807	4.12×	0.000168	2.75×
8,000	0.031517	4.04×	0.000356	2.12×
18,250	0.174504	5.54×	0.000932	2.62×
32,000	0.606400	3.47×	0.001701	1.83×
64,000	2.950611	4.87×	0.003499	2.06×
128,000	13.437407	4.55×	0.007202	2.06×
256,000	57.313804	4.27×	0.015157	2.10×
500,000	—	—	0.029352	1.94×
1,000,000	871.760620	15.21×	0.070114	2.39×
2,000,000	—	—	0.154128	2.20×
4,000,000	—	—	0.352225	2.29×
8,000,000	—	—	0.879219	2.50×

(Bubble sort ratios spanning a non-doubling step — 8,000 → 18,250 and 256,000 → 1,000,000 — are larger because n more than doubles there.)

Bubble Sort — quadratic, confirmed. Every doubling of n multiplies time by ≈ 4 (measured 4.04–4.87 across the clean doubling steps). The strongest single check is the last step: 256,000 $\rightarrow$ 1,000,000 is 3.906 $\times$ more data, and pure n^2 predicts a **15.26 $\times$** slowdown. Measured: **15.21 $\times$. 871.8 seconds** to sort one million records.

Quick Sort — $n \log n$, confirmed. Every doubling multiplies time by ≈ 2.1 , against a theoretical 2.11 $\times$ in this range. The two smallest sizes (1,000 and 2,000) show inflated ratios only because 17–61 μ s is near the timer’s noise floor.

Beyond ~ 2 M elements the ratio drifts above prediction (2.29 $\times$, 2.50 $\times$ versus $\sim 2.09\times$ predicted). At 8 M elements the array is **32 MB**, well past the 12 MB L3 cache, so memory traffic — not comparison count — starts to set performance. This is exactly the regime where cache behaviour matters more than asymptotics.

The gap widens without limit. The ratio is not a constant factor; it grows as $O(n / \log n)$:

N	Bubble	Quick	Bubble / Quick
1,000	0.000424 s	0.000017 s	25 $\times$
18,250	0.174504 s	0.000932 s	187 $\times$
256,000	57.31 s	0.015157 s	3,781 $\times$
1,000,000	871.76 s	0.070114 s	12,433$\times$

10. Feasibility of parallel execution

Implemented as OpenMP tasks (quickSortOMP in mysort.c, build with -fopenmp). The two recursive calls touch disjoint halves of the array, so they are independent and safe to run concurrently. Two guards prevent collapse under task-creation overhead: stop spawning below a depth of $\sim \log_2(\text{threads})$, and sort subarrays under 50,000 elements serially.

Measured at $N = 4,000,000$ on 12 threads:

Version	Wall (s)	CPU (s)	Speedup
Serial quicksort	0.5537	0.5560	1.0 $\times$
OpenMP quicksort	0.2267	1.7019	2.4$\times$

Verdict: worthwhile but sharply limited. 2.4 $\times$ on 12 threads is 20% efficiency. The reason is structural, not an implementation defect:

- **The first partition is inherently serial.** It touches all n elements before any parallelism exists. That alone is $\sim 50\%$ of the total comparison work at the top two levels, and by Amdahl’s law a 50% serial fraction caps speedup at 2 $\times$ regardless of core count. The measured 2.4 $\times$ is consistent with this.
- **Memory bandwidth saturates.** Sorting is memory-bound at 16 MB; extra threads compete for the same L3 and DRAM bandwidth.
- **CPU time rose 3.1 $\times$** (0.556 $\rightarrow$ 1.702 s) — parallelism bought latency at a real cost in total work, which matters on shared or battery-powered machines.
- This CPU is 2 performance cores + 8 efficient cores, so 12 threads are not 12 equal cores.

For genuinely large datasets, a parallel **merge** sort or sample sort scales better, because it parallelises the partitioning step itself rather than only the recursion. **Bubble sort is not usefully parallelisable** in this form — each pass depends on the previous one. Its parallel relative, odd-even transposition sort, still needs $O(n)$ parallel steps and remains far worse than serial quicksort.

11. Answers to the lab questions

1. Which sorting algorithm performed better? Explain. Quick Sort, overwhelmingly — **187×** at $N = 18,250$ and **12,433×** at $N = 1,000,000$. The cause is algorithmic complexity, not implementation quality: bubble sort is $O(n^2)$ and performed 166,521,222 comparisons on the 18,250-record dataset, while quicksort is $O(n \log n)$ and performed 302,516 — a **550×** difference in work. Bubble sort compares only adjacent elements, so each swap moves an element one position; quicksort's partitioning moves elements a long way toward their final position, so it never revisits the same pair.

2. Which function consumed the maximum execution time? For the baseline, bubbleSort at **73.53%** of CPU time, with swap second at **23.53%** across **82,919,103 calls**. For quicksort (profiled at $N = 4,000,000$ so gprof can resolve it), partition at **78.57%** — expected, since partition is where every comparison happens. At -O2 swap disappears from the profile entirely because it is inlined; at -O3 even bubbleSort is inlined into its caller.

3. How does execution time change with increasing input size? Bubble sort quadruples per doubling of n (measured 4.04–4.87×, theory 4×); quicksort roughly doubles (measured ~2.1×, theory 2.11×). The sharpest check: going from 256,000 to 1,000,000 records, n^2 predicts a 15.26× slowdown and the measurement gives 15.21×. The gap therefore widens without bound — 25× at $N = 1,000$ but **12,433×** at $N = 1,000,000$. Beyond ~2 M elements quicksort drifts slightly above its $n \log n$ prediction as the working set (32 MB at $N = 8$ M) exceeds the 12 MB L3 cache and memory bandwidth becomes the limit.

4. How did compiler optimizations (-O2 and -O3) improve performance? -O2 gave a **2.03×** speedup on bubble sort, almost entirely by inlining swap — eliminating 82.9 M function calls, which is visible as swap disappearing from the -O2 gprof report. -O3 **made it 3.1×** worse (0.166 s → 0.520 s): it enables SLP auto-vectorization, which emits xmm registers and pshufd shuffles into a loop that has a loop-carried dependency. -fno-tree-vectorize recovers the loss, confirming the cause. Compiler optimization is worth ~2×; the algorithm change is worth ~10,000×.

5. Which algorithm would you recommend for large datasets? Justify. **Quick Sort**, with the reservations below. At 1,000,000 records it finishes in 0.070 s against bubble sort's 871.8 s — the difference between interactive and unusable. Bubble sort should be retained only as a teaching baseline.

Caveats worth stating rather than hiding:

- The supplied Lomuto partition takes the **last element as pivot**, which degrades to **$O(n^2)$ time and $O(n)$ recursion depth on already-sorted or reverse-sorted input** — and weather data very often arrives sorted by date. A production version needs median-of-three or a randomized pivot, plus recursion on the smaller side first to bound stack depth.
- Quicksort is **not stable**. If records carry more than a temperature (station ID, timestamp), equal-key ordering is not preserved.
- For the actual business problem — **18,250 records, sorted in under 1 ms** — the honest recommendation is `qsort()` from `libc`, or `std::sort`. It is already written, already tested, and uses introsort, which falls back to heap sort when quicksort's recursion goes bad. Writing a custom quicksort is only justified if profiling proves the library call is the bottleneck, and at 0.9 ms it is not.

12. Recommendations

1. **Replace bubble sort with quicksort** — 187× today, and the margin grows with the dataset.
2. **Ship with -O2, not -O3.** -O3 is measurably slower on this code. Treat -O3 as a hypothesis to be benchmarked, never a default.

3. **Fix the pivot before production.** Last-element pivot on date-ordered weather data is a latent $O(n^2)$ and a stack-overflow risk.
4. **Prefer `qsort()`** unless profiling proves otherwise.
5. **Parallelise only above ~1 M records.** Below that, serial quicksort's sub-millisecond runtime makes 2.4× irrelevant.
6. **Profile at -02, not -00 or -03.** -00 inflates call overhead by 2.55×; -03 inlines the functions you are trying to observe.